

 SAKARYA UNİVERSİTESİ	T.C. SAKARYA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ UYGULAMALI MÜHENDİSLİK DENEYİMİ EĞİTİMİ (UMDE)	
---	---	--

UMDE ADAY MÜHENDİS FAALİYET ARA RAPORU (F-PG-1)

Aday Mühendisin Adı ve Soyadı	ATAKAN ÇALIŞKAN
UMDE İşletmesinin Adı	TEI-TUSAŞ MOTOR SANAYİİ A.Ş.
Çalıştığı Bölüm ve Alan	Yazılım Müdürlüğü – Gömülü Yazılım Geliştirme
Teslim Edilen Ara Rapor No	6
Faaliyet Zaman Aralığı	23 / 03 /2026 - 26 / 03 /2026

UMDE Aday Mühendis; faaliyet ara raporunu danışman öğretim üyesi/görevlisine her hafta teslim etmek zorundadır. Akademik danışman bu rapora göre değerlendirme yapacaktır.

FAALİYET ARA RAPORU
1.GENEL C ÇALIŞMALARI Bu hafta gerçekleştirilen genel C çalışmaları kapsamında, gömülü yazılım geliştirme sürecinin yalnızca kaynak kod yazmaktan ibaret olmadığı; derleme, bağlama, bellek yerleşimi ve çıktı dosyalarının oluşturulması gibi birden fazla teknik aşamadan oluştuğu değerlendirilmiştir. Bu doğrultuda compiler yapısı, optimizasyon mantığı, assembly kod ilişkisi, linker'ın görevleri, bellek bölümlerinin yerleşimi, yürütülebilir dosya formatları ve preprocessor yapısı üzerinde durulmuştur. 1.1 Derleme süreci, compiler ve optimizasyon kavramı Yapılan çalışmalarda öncelikle compiler kavramı ele alınmıştır. Compiler, C dilinde yazılmış kaynak kodu makine tarafından işlenebilecek daha alt seviyeli bir forma dönüştüren araçtır. Bu süreçte yalnızca çeviri yapılmamakta; aynı zamanda sözdizimi denetimi gerçekleştirilmekte, tanımsız yapılar kontrol edilmekte ve hedef işlemciye uygun çıktı üretilmektedir. Böylece yazılan kaynak kod, donanım üzerinde çalıştırılabilecek hale getirilmektedir. Bu kapsamda optimizasyon kavramı da incelenmiştir. Compiler'ın görevi yalnızca doğru çalışan kod üretmek değil, aynı zamanda daha hızlı çalışan, daha az yer kaplayan veya daha verimli çalışan çıktı üretebilmektir. Bu amaçla farklı optimizasyon seviyeleri kullanılmaktadır. Optimizasyon kapalı olduğunda kod daha doğrudan çevrilmekte ve debug süreci daha kolay takip edilebilmektedir. Optimizasyon seviyesi arttıkça ise gereksiz işlemler kaldırılabilmekte, kod sıralaması değiştirilebilmekte ve daha verimli makine kodu elde edilebilmektedir. Buna karşılık yüksek

optimizasyon seviyelerinde debug süreci zorlaşabilmekte, bazı değişkenler beklenen şekilde gözlemlenememekte ve kaynak kod ile çalışan çıktı arasındaki doğrudan ilişki zayıflayabilmektedir. Bu nedenle geliştirme ve hata ayıklama sürecinde optimizasyon seviyesinin bilinçli seçilmesi gerektiği görülmüştür.

1.2 Assembly kodu ile ilişkisi

Compiler çalışmaları kapsamında C kodu ile **assembly kodu** arasındaki ilişki de değerlendirilmiştir. C dili ile yazılan programlar, derleme sürecinde işlemci mimarisine uygun assembly ve sonrasında makine koduna dönüştürülmektedir. Assembly, makine diline insan tarafından daha okunabilir bir ara seviye temsil sunmaktadır. Register kullanımı, bit işlemleri, veri taşıma ve düşük seviyeli kontrol adımları assembly seviyesinde daha açık biçimde görülebilmektedir.

Bu nedenle bazı durumlarda assembly çıktısının incelenmesine ihtiyaç duyulmaktadır. Özellikle yazılan C kodu beklenenden farklı çalıştığında, compiler optimizasyonunun nasıl etki ettiğini anlamak gerektiğinde veya donanım çok yakın register/bit işlemleri takip edilmek istendiğinde assembly incelemesi önemli hale gelmektedir. Böylece CPU'nun gerçekte hangi komutları yürüttüğü daha net görülebilmektedir. Bu çalışmalarda compiler'ın neden assembly ürettiği, assembly'nin neden gerekli olduğu ve düşük seviyeli davranışın bu katmanda nasıl gözlemlenebildiği değerlendirilmiştir.

1.3 Linker yapısı ve nesne dosyalarının birleştirilmesi

Bu hafta üzerinde durulan bir diğer önemli başlık **linker** olmuştur. Derleme sonrasında her kaynak dosya doğrudan tek başına çalışan bir program oluşturmamakta; önce ayrı ayrı nesne dosyalarına dönüştürülmektedir. Linker'ın temel görevi bu farklı nesne dosyalarını bir araya getirerek tek bir bütün program oluşturmaktır. Bu süreçte fonksiyon çağrılarını eşleştirmekte, global değişkenler çözülmekte ve programın hangi bölümünün bellekte nereye yerleşeceği belirlenmektedir.

Birden fazla nesne dosyasının birleşmesi sırasında sembol çözümleme işlemi yapılmakta, bir dosyada çağrılan fonksiyonun diğer dosyada bulunan tanımı ile eşleştirme sağlanmaktadır. Aynı şekilde veri alanları da uygun bellek bölgelerine yerleştirilmektedir. Bu nedenle linker yalnızca dosyaları yan yana ekleyen bir yapı değil; programın bellek içi son düzenini oluşturan kritik bir aşama olarak değerlendirilmiştir.

1.4 Bellek bölümleri ve yerleşim mantığı

Linker ile doğrudan ilişkili olarak **global değişkenlerin bellekte nasıl yerleştirildiği** ve yazılım bölümlerinin hangi alanlarda tutulduğu da incelenmiştir. Bu kapsamda .text, .data, .bss ve .rodata bölümleri üzerinde durulmuştur. Program kodu .text bölümünde, sabit veriler .rodata bölümünde,

başlangıç değeri verilmiş global değişkenler .data bölümünde, başlangıç değeri verilmeyen global değişkenler ise .bss bölümünde yer almaktadır.

Bu yapıdan hareketle, flash ve RAM arasındaki görev ayrımı daha net anlaşılmıştır. Program kodu ve kalıcı sabit bilgiler genellikle flash içinde tutulurken, çalışma sırasında aktif olarak kullanılan değişkenler RAM tarafında yer almaktadır. Özellikle .data ve .bss bölümlerinin başlangıçta uygun şekilde hazırlanması, startup süreci ile de doğrudan bağlantılıdır. Bu çalışma sayesinde gömülü yazılımda kod yazmanın yanında, kodun bellekte nasıl yerleştiğinin de kritik olduğu görülmüştür.

1.5 Linker script, stack ve heap yerleşimi

Bellek yerleşimi ile bağlantılı olarak **linker script** yapısı da değerlendirilmiştir. Linker script, programın hangi bölümünün hangi bellek adresine yerleşeceğini tanımlayan dosyadır. Başka bir ifadeyle .text, .data, .bss, stack ve heap gibi alanların hangi adres aralıklarında bulunacağını linker script belirlemektedir. Bu dosya sayesinde programın doğru bellek bölgelerinde çalışması sağlanmaktadır.

Bu kapsamda **stack ve heap'in linker script içindeki yeri** de ayrıca ele alınmıştır. Her iki alan da RAM içinde yer almakla birlikte, farklı yönlerden büyüebilmektedir. Stack genellikle RAM'in üst tarafına yakın bir noktadan başlayıp aşağı doğru büyürken, heap daha alt bölgelerden başlayıp yukarı doğru büyümektedir. Bu sınırların yanlış belirlenmesi durumunda bellek çakışmaları ve sistem hataları oluşabilmektedir. Bu nedenle stack ve heap yerleşiminin yalnızca yazılım açısından değil, sistem güvenilirliği açısından da önemli olduğu görülmüştür.

1.6 Çıktı dosyaları: ELF, HEX ve BIN

Derleme ve linkleme sürecinin sonunda elde edilen çıktı dosyaları da bu hafta incelenmiştir. ELF dosyası, linkleme işlemi tamamlanmış, çalıştırılabilir program bilgisini içeren ve debug için de zengin bilgi taşıyan bir dosya yapısıdır. Geliştirme ortamlarında genellikle yükleme ve hata ayıklama süreçlerinde bu format kullanılmaktadır.

Buna karşılık HEX ve BIN dosyaları daha çok hedef cihaza program yükleme amacıyla öne çıkmaktadır. BIN dosyası ham makine kodunu içerirken, HEX dosyası adres bilgisini de taşıyan metin tabanlı bir formattır. Bu nedenle özellikle flash belleğe doğru adreslerden program yüklenmesi gerektiğinde HEX formatı avantaj sağlamaktadır. Bu çalışma ile farklı çıktı dosyalarının yalnızca uzantı farkı taşımadığı, her birinin derleme-yükleme zinciri içinde farklı teknik amaçlara hizmet ettiği anlaşılmıştır.

1.7 Preprocessor yapısı

Son olarak **preprocessor** kavramı üzerinde durulmuştur. Preprocessor, derlemeden önce çalışan ve kaynak kod üzerinde ön işlem yapan aşamadır. #include, #define, koşullu derleme yapıları ve makro işlemleri bu aşamada değerlendirilmektedir. Böylece kaynak dosyalar birleştirilmekte, sabit tanımlar genişletilmekte ve bazı kod parçaları derleme öncesinde şekillendirilmektedir.

Bu yapının özellikle modüler kod geliştirme açısından önemli olduğu görülmüştür. Başlık dosyalarının eklenmesi, sembolik sabitlerin tanımlanması ve belirli koşullarda farklı kod yollarının seçilmesi preprocessor üzerinden sağlanmaktadır. Bu nedenle preprocessor, derleyiciden önce çalışan ancak yazılımın son yapısını doğrudan etkileyen önemli bir hazırlık katmanı olarak değerlendirilmiştir.

2.RTC İLE C2000 SERİSİ MİKROKONTROLÇÜ ARASINDA GPIO ÜZERİNDEN HABERLEŞME AKIŞI

Bu hafta geliştirilen yazılım yapısında, DS1302 gerçek zaman saati modülü ile C2000 serisi mikrodenetleyici arasında haberleşme, doğrudan çevre birimi tabanlı bir haberleşme modülü kullanılmadan, GPIO pinleri üzerinden yazılımsal olarak oluşturulmuştur. Bu yaklaşımda RTC modülünün **RST**, **DAT** ve **CLK** hatları mikrodenetleyicinin dijital pinleri ile kontrol edilmiş; veri aktarımı saat darbesi üretilerek ve veri hattı yönü yazılım tarafından değiştirilerek gerçekleştirilmiştir.

Geliştirilen yapı üç katmanlı olacak şekilde düzenlenmiştir. **HAL katmanı**, pin yönü değiştirme, pin seviyelerini yazma ve okuma gibi doğrudan donanım erişim işlemlerini üstlenmektedir. **BSP katmanı**, DS1302'ye özgü yazma-okuma akışını ve başlangıç konfigürasyonlarını gerçekleştirmektedir. **APP katmanı** ise uygulama seviyesinde saat verisinin okunması, BCD formatındaki bilginin decimal formata çevrilmesi ve ana program akışının yürütülmesini sağlamaktadır. Bu katmanlı yapı sayesinde hem kodun okunabilirliği artırılmış hem de ilerleyen aşamalarda daha sürdürülebilir ve modüler bir mimariye geçiş için uygun bir temel oluşturulmuştur.

RTC ile GPIO üzerinden gerçekleştirilen haberleşme akışı genel olarak aşağıdaki adımlardan oluşmaktadır:

1. **Sistem başlatma işlemleri gerçekleştirilir.**
Mikrodenetleyicinin temel sistem ve GPIO altyapısı hazırlanır.
2. **RTC haberleşmesinde kullanılacak pinler yapılandırılır.**
RST ve CLK pinleri çıkış olarak ayarlanır. DAT pini ise başlangıçta çıkış olarak tanımlanır.
3. **RTC modülünün ilk konfigürasyonu yapılır.**
Yazma koruması kaldırılır, saniye, dakika ve saat register'larına başlangıç değerleri yazılır.
4. **Yazma işlemi başlatılırken RST hattı aktif edilir.**
RTC ile haberleşmenin başlangıcı için RST hattı lojik 1 yapılır.
5. **Adres ve veri bilgisi CLK darbesi ile seri olarak gönderilir.**
Yazma sırasında adres ve veri byte'ları, DAT hattına bit bit yerleştirilir ve her bit için CLK hattı üzerinden darbe üretilir.
6. **Okuma işlemi sırasında veri hattının yönü değiştirilir.**
RTC'ye okuma komutu gönderildikten sonra DAT pini çıkış modundan giriş moduna alınır.
7. **RTC'den veri seri olarak okunur.**
DAT hattındaki bit bilgisi her clock çevriminde okunur ve 8 bitlik sonuç oluşturulur.
8. **Okuma tamamlandıktan sonra veri hattı tekrar çıkış moduna alınır.**
Böylece bir sonraki yazma veya okuma işlemi için pin yapısı yeniden hazırlanır.
9. **Haberleşme çevrimi sonlandırılır.**
RST hattı lojik 0 yapılarak RTC ile olan seri erişim tamamlanır.
10. **RTC'den alınan saat bilgileri BCD formatından decimal formata çevrilir.**
Okunan saniye, dakika ve saat register içerikleri uygulama seviyesinde işlenir.
11. **Ana döngü içinde belirli aralıklarla saat verisi okunur.**
Böylece RTC modülündeki zaman bilgisi sürekli izlenebilir hale gelir.

Bu akış incelendiğinde, RTC haberleşmesinin temel olarak üç ana aşamada gerçekleştiği görülmektedir: pinlerin hazırlanması, seri veri aktarımının yazılımsal olarak **oluşturulması** ve okunan zaman bilgisinin uygulama seviyesinde işlenmesi. Bu yaklaşım, haberleşme protokolünün düşük seviyede anlaşılması açısından faydalı olmakla birlikte, aynı zamanda DS1302 gibi özel zaman tutma

entegreleriyle çevre birimi kullanmadan doğrudan GPIO kontrolü üzerinden çalışılabileceğini göstermektedir.

Bu kod yapısına göre sistemde doğrudan bir I2C veya SPI çevre birimi kullanılmamış, bunun yerine GPIO pinleri üzerinden yazılımsal saatleme ile haberleşme gerçekleştirilmiştir. Veri hattının hem yazma hem de okuma için kullanılması nedeniyle pin yönünün dinamik olarak değiştirilmesi kritik bir adımdır. Ayrıca yazılım yapısının **HAL, BSP ve APP** katmanlarına ayrılmış olması, donanım erişimi ile uygulama mantığının birbirinden ayrılmasını sağlayarak kodun ilerleyen aşamalarda geliştirilmesini kolaylaştırmaktadır.

3.HAL Katmanı

Kullanılan Kod	Kullanım Amacı	Etkilediği Register / Bit
<code>#define RST_PIN 13</code>	RTC haberleşmesinde kullanılacak reset/enable hattının pin numarasını tanımlar.	Doğrudan register değiştirmez; sonraki GPIO işlemlerinde GPIO13 bitini hedef olarak belirler.
<code>#define DAT_PIN 14</code>	RTC veri hattı olarak kullanılacak pin numarasını tanımlar.	Doğrudan register değiştirmez; sonraki işlemlerde GPIO14 bitini hedef olarak belirler.
<code>#define CLK_PIN 15</code>	RTC clock hattı olarak kullanılacak pin numarasını tanımlar.	Doğrudan register değiştirmez; sonraki işlemlerde GPIO15 bitini hedef olarak belirler.
<code>GPIO_setDirectionMode(RST_PIN, GPIO_DIR_MODE_OUT);</code>	RST hattını çıkış yapar. Mikrodenetleyici bu pini sürmeye başlar.	GPIO yön register'ı, ilgili pin yön biti: işlevsel olarak GPIO13 direction bit
<code>GPIO_setDirectionMode(CLK_PIN, GPIO_DIR_MODE_OUT);</code>	CLK hattını çıkış yapar. Clock darbeleri yazılımla üretilebilsin diye kullanılır.	GPIO yön register'ı, ilgili pin yön biti: GPIO15 direction bit
<code>GPIO_setDirectionMode(DAT_PIN, GPIO_DIR_MODE_OUT);</code>	Veri hattını başlangıçta çıkış yapar. Yazma işlemlerinde veri hattı mikrodenetleyici tarafından sürülür.	GPIO yön register'ı, ilgili pin yön biti: GPIO14 direction bit

 SAKARYA ÜNİVERSİTESİ	T.C. SAKARYA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ UYGULAMALI MÜHENDİSLİK DENEYİMİ EĞİTİMİ (UMDE)	
---	--	--

<code>GPIO_writePin(RST_PIN, 0);</code>	RTC haberleşmesini pasif durumda başlatır. Başlangıçta DS1302 seçili olmaz.	GPIO veri çıkış register'ı, ilgili çıkış biti: GPIO13 output/data bit
<code>GPIO_writePin(CLK_PIN, 0);</code>	Clock hattını başlangıçta düşük seviyeye çeker.	GPIO veri çıkış register'ı, ilgili çıkış biti: GPIO15 output/data bit
<code>if(is_input)</code>	Veri hattının yönünü çalışma anına göre değiştirmek için karar yapısıdır.	Doğrudan register değiştirmez; aşağıdaki giriş/çıkış yapılandırmasını seçer.
<code>GPIO_setPadConfig(DAT_PIN, GPIO_PIN_TYPE_PULLUP);</code>	DAT hattı giriş yapılırken pull-up ile kararlı hale getirilir. RTC veri okumasında hat boşta kalmaz.	GPIO pad/pull-up konfigürasyon register alanı, ilgili bit: GPIO14 pad/pull-up ayarı
<code>GPIO_setDirectionMode(DAT_PIN, GPIO_DIR_MODE_IN);</code>	Veri hattını giriş yapar. Böylece RTC'den gelen veri okunabilir.	GPIO yön register'ı, ilgili pin yön biti: GPIO14 direction bit = input
<code>GPIO_setPadConfig(DAT_PIN, GPIO_PIN_TYPE_STD);</code>	DAT hattını yeniden standart çıkış yapısına döndürür. Yazma öncesi normal pin sürme davranışı sağlanır.	GPIO pad konfigürasyon register alanı, ilgili bit: GPIO14 pad type ayarı
<code>GPIO_setDirectionMode(DAT_PIN, GPIO_DIR_MODE_OUT);</code>	Veri hattını tekrar çıkış yapar. Böylece mikrodenetleyici RTC'ye veri yazabilir.	GPIO yön register'ı, ilgili pin yön biti: GPIO14 direction bit = output
<code>GPIO_writePin(DAT_PIN, (data >> i) & 0x01);</code>	Gönderilecek byte'ın ilgili bitini DAT hattına yazar. DS1302'ye seri veri aktarımı yapılır.	GPIO veri çıkış register'ı, ilgili bit: GPIO14 output/data bit
<code>GPIO_writePin(CLK_PIN, 1);</code>	Clock hattını 1 yaparak RTC'ye veri örnekleme darbesi üretir.	GPIO veri çıkış register'ı, ilgili bit: GPIO15 output/data bit = high
<code>DEVICE_DELAY_US(20);</code>	Clock darbesinin zamanlamasını oluşturur. RTC'nin veri örnekleyebilmesi için kısa bekleme eklenir.	Doğrudan GPIO register'ı değil; sistem delay/zamanlama altyapısını kullanır.
<code>GPIO_writePin(CLK_PIN, 0);</code>	Clock hattını tekrar 0 yaparak bir clock çevrimini tamamlar.	GPIO veri çıkış register'ı, ilgili bit: GPIO15 output/data bit = low

 SAKARYA ÜNİVERSİTESİ	T.C. SAKARYA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ UYGULAMALI MÜHENDİSLİK DENEYİMİ EĞİTİMİ (UMDE)	
--	--	--

<code>if(GPIO_readPin(DAT_PIN))</code>	RTC'den gelen bit bilgisini veri hattından okur.	GPIO giriş veri register'ı, ilgili bit: GPIO14 input/data bit
<code>data</code>	<code>= (1 << i);</code>	Okunan bit '1' ise bunu yazılım içindeki data değişkenine yerleştirir.
<code>return data;</code>	Okunan 8 bit veriyi üst katmana döndürür.	Donanım register'ı değil; fonksiyon dönüş değeridir.

4.BSP Katmanı

Kullanılan Kod	Kullanım Amacı	Etkilediği Register / Bit
<code>HAL_DS1302_Init();</code>	RTC haberleşmesinde kullanılacak RST, DAT ve CLK pinlerinin başlangıç durumlarını hazırlar. BSP katmanının kullanacağı düşük seviyeli donanım altyapısını başlatır.	DS1302 register'ı değil; C2000 tarafında GPIO13, GPIO14, GPIO15 yön ve veri bitleri etkilendir.
<code>RTC_Yaz(0x8E, 0);</code>	DS1302'nin yazma korumasını kaldırır. RTC register'larına yazılabilir duruma geçilir.	DS1302 Control Register, yazma adresi 0x8E; kritik bit: WP (Write Protect), bit 7 = 0
<code>RTC_Yaz(0x80, 0);</code>	Saniye register'ına başlangıç değeri yazar. Aynı zamanda saat osilatörünün çalışması sağlanır.	DS1302 Seconds Register, yazma adresi 0x80; kritik bit: CH (Clock Halt), bit 7 = 0
<code>RTC_Yaz(0x82, 0);</code>	Dakika register'ını başlangıçta 00 yapar.	DS1302 Minutes Register, yazma adresi 0x82
<code>RTC_Yaz(0x84, 1);</code>	Saat register'ına başlangıç değeri yazar. Sistem ilk başlatmada saat bilgisini oluşturur.	DS1302 Hours Register, yazma adresi 0x84
<code>GPIO_writePin(RST_PIN, 1);</code>	RTC ile haberleşmeyi başlatır. DS1302'nin komut kabul etmesi için CE/RST hattı aktif edilir.	C2000 tarafında GPIO13 output bit; DS1302 tarafında haberleşme çevrimi aktif hale gelir

 SAKARYA ÜNİVERSİTESİ	T.C. SAKARYA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ UYGULAMALI MÜHENDİSLİK DENEYİMİ EĞİTİMİ (UMDE)	
---	---	--

<code>DEVICE_DELAY_US(20);</code>	RST hattı aktif olduktan sonra zamanlama kararlılığı sağlamak için kısa bekleme ekler.	Doğrudan register değil; sistem zamanlama altyapısı
<code>HAL_DS1302_WriteByte(adres);</code>	RTC'ye önce hedef register adresini gönderir. Hangi register'a erişileceği belirlenir.	DS1302 tarafında gönderilen komut byte'ı; hedefe göre Seconds, Minutes, Hours, Control vb. register seçilir
<code>HAL_DS1302_WriteByte(veri);</code>	Seçilen RTC register'ına veri yazar.	DS1302'de adrese karşılık gelen register'ın veri alanı güncellenir
<code>GPIO_writePin(RST_PIN, 0);</code>	Yazma çevrimini sonlandırır. RTC ile olan aktif haberleşme kapatılır.	C2000 tarafında GPIO13 output bit = 0
<code>uint16_t sonuc;</code>	RTC'den okunacak veriyi geçici olarak tutar.	Donanım register'ı değil; RAM üzerindeki yerel değişken
<code>HAL_DS1302_PinDir(true);</code>	Adres gönderildikten sonra DAT hattını giriş moduna alır. RTC'nin veri hattını sürmesine izin verir.	C2000 tarafında GPIO14 direction bit = input, pad config değişir
<code>sonuc = HAL_DS1302_ReadByte();</code>	RTC register'ından gelen 8 bit veriyi okur.	Hedef DS1302 register'ının içeriği okunur; fiziksel olarak C2000 tarafında GPIO14 input bit kullanılır
<code>HAL_DS1302_PinDir(false);</code>	Okuma sonrası DAT hattını tekrar çıkış moduna alır. Sonraki yazma/komut işlemleri için hattı mikrodenetleyici sürer.	C2000 tarafında GPIO14 direction bit = output, pad config değişir
<code>return sonuc;</code>	Okunan RTC verisini üst katmana döndürür.	Donanım register'ı değil; fonksiyon dönüş değeri

4. APP(MAIN) KATMANI

Kullanılan Kod	Kullanım Amacı	Etkilediği Register / Bit
#include "driverlib.h"	GPIO, sistem ve temel donanım fonksiyonlarını kullanmak için gerekli kütüphaneyi projeye dahil eder.	Doğrudan register değiştirmez; Driverlib erişim altyapısını sağlar.
#include "device.h"	Cihaz başlatma ve sistem altyapısı fonksiyonlarını projeye dahil eder.	Doğrudan register değiştirmez; sistem başlatma altyapısını sağlar.
#include "bsp_ds1302.h"	RTC'ye özgü BSP fonksiyonlarını uygulama katmanına dahil eder.	Doğrudan register değiştirmez; BSP arayüzünü görünür kılar.
uint16_t saniye_dec, dakika_dec, saat_dec;	RTC'den okunacak zaman bilgisinin decimal karşılığını saklar.	Donanım register'ı değil; RAM üzerinde global değişkenlerdir.
uint32_t loop_sayaci = 0;	Ana döngünün çalıştığını kontrol etmek ve debug sürecinde akışı izlemek için kullanılır.	Donanım register'ı değil; RAM üzerinde global değişkendir.
Device_init();	Mikrodenetleyicinin temel sistem başlatma işlemlerini gerçekleştirir.	Sistem clock ve temel cihaz altyapısı
Device_initGPIO();	GPIO altyapısını başlatır ve pin kullanımına hazır hale getirir.	GPIO alt sistemi
BSP_DS1302_Config();	RTC'nin başlangıç konfigürasyonunu yapar; yazma korumasını kaldırır ve temel saat bilgilerini yazar.	DS1302 tarafında Control Register (WP bit), Seconds Register (CH bit), Minutes Register, Hours Register
while(1)	Uygulamanın sürekli çalışmasını sağlayan ana döngüdür. RTC verileri periyodik olarak bu döngü içinde okunur.	Doğrudan register değiştirmez; sürekli uygulama akışını oluşturur.
uint16_t s = RTC_Oku(0x81);	RTC'den saniye bilgisini okur.	DS1302 Seconds Register, okuma adresi 0x81
uint16_t m = RTC_Oku(0x83);	RTC'den dakika bilgisini okur.	DS1302 Minutes Register, okuma adresi 0x83
uint16_t h = RTC_Oku(0x85);	RTC'den saat bilgisini okur.	DS1302 Hours Register, okuma adresi 0x85
saniye_dec = (s & 0x0F) + ((s >> 4) * 10);	BCD formatındaki saniye bilgisini decimal formata çevirir.	Donanım register'ı değil; okunan saniye verisi RAM üzerinde işlenir.
dakika_dec = (m & 0x0F) + ((m >> 4) * 10);	BCD formatındaki dakika bilgisini decimal formata çevirir.	Donanım register'ı değil; okunan dakika verisi RAM üzerinde işlenir.

 SAKARYA UNİVERSİTESİ	T.C. SAKARYA ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ UYGULAMALI MÜHENDİSLİK DENEYİMİ EĞİTİMİ (UMDE)	
---	---	--

<code>saat_dec = (h & 0x0F) + ((h >> 4) * 10);</code>	BCD formatındaki saat bilgisini decimal formata çevirir.	Donanım register'ı değil; okunan saat verisi RAM üzerinde işlenir.
<code>loop_sayaci++;</code>	Ana döngünün her çevrimde ilerlediğini göstermek için sayaç artırılır.	Donanım register'ı değil; RAM üzerinde yazılım değişkenidir.
<code>DEVICE_DELAY_US(1000000);</code>	Her okuma çevrimi arasına yaklaşık 1 saniyelik gecikme koyar.	Doğrudan GPIO/RTC register'ı değil; sistem delay altyapısını kullanır.

Sonuç olarak bu katmanlı yapı sayesinde RTC modülü ile gerçekleştirilen GPIO tabanlı haberleşme; düşük seviyeli pin kontrolü, RTC'ye özgü okuma-yazma akışı ve uygulama seviyesinde zaman bilgisinin işlenmesi olmak üzere üç ayrı seviyede düzenli biçimde yapılandırılmış; böylece hem modüler hem de sürdürülebilir bir yazılım altyapısı oluşturulmuştur.

Aday Mühendisin İmzası	UMDE Sorumlusu İmzası	Danışman Öğretim Üyesi/Görevlisi İmzası